

Comparative Analysis of Methods for Performing a Side-Channel Video-Fingerprinting Attack

1st Deborah Djon

*School of Computing
Dublin City University*

Collins Ave Ext, Whitehall, Dublin 9
deborah.djon3@mail.dcu.ie

2nd Darragh Connaughton

*School of Computing
Dublin City University*

Collins Ave Ext, Whitehall, Dublin 9
darragh.connaughton5@mail.dcu.ie

3rd Geoff Hamilton

*School of Computing
Dublin City University*

Collins Ave Ext, Whitehall, Dublin 9
geoff.hamilton@dcu.ie

4th Andrew McCarren

*Insight (12/RC/2289_p2), School of Computing
Dublin City University*

Collins Ave Ext, Whitehall, Dublin 9
andrew.mccarren@dcu.ie

Abstract—In an era of increasing digital privacy concerns, side-channel attacks pose a risk to the privacy of many streaming service clients. With the attack outlined in this study, it is possible to detect what YouTube video a user is watching despite network traffic encryption. An attacker can achieve this by performing a side-channel attack that targets the byte sizes of the Hypertext Transfer Protocol Secure (HTTPS) requests transmitted between the user’s browser and the YouTube server. This study investigates the feasibility of detecting YouTube videos using HTTPS-packet-related network data captured from application logs. Convolutional Neural Networks (CNN), logistic regression, and Dynamic Time Warping (DTW) were used for identifying fingerprinted YouTube videos with side-channel data in an open-world scenario, where most of the examples the model is presented with are unknown. Our research shows that it is possible to identify YouTube videos using network traces with an F1-score of 80% in an open-world scenario using a CNN approach. We further show that it is possible to distinguish known and unknown videos with a recall of 92% in the same open-world scenario. We also show that similar results could not be attained using logistic regression or DTW proposed by other authors.

Index Terms—SCA, machine learning, cyber security, CNN, timeseries

I. INTRODUCTION

Recent studies reveal a risk to streaming content consumer’s privacy. User privacy and sensitive content on the internet can be protected from unauthorised entities through, network traffic encryption technologies such as the Hypertext Transfer Protocol Secure (HTTPS), Virtual Private Networks (VPNs), and The Onion Router (TOR). Major streaming providers use the streaming content delivery technique Dynamic Adaptive Streaming over HTTP (DASH). The use of DASH improves a user’s Quality of Experience (QoE) by reducing stalls at video play time [1]. However, recent works suggest that DASH leaves a unique pattern in streaming traffic that can be extracted and used to identify specific videos, [2], [3], [4], [5]. Although network encryption renders video identification through Deep Packet Inspection (DPI) ineffective, videos can

still be identified by extracting patterns in the sizes of packets traversing the network [2], [6]. Stakeholders, such as Internet Service Providers (ISPs) or universities can leverage a Side Channel Attack (SCA) on encrypted network traces to identify and restrict access to specific content within their networks. This paper delves into the feasibility of the SCA on HTTPS packet-related data. The primary contributions of this study are as follows:

- A comparison of logistic regression, dynamic Time Warping (DTW), and Convolutional Neural Network (CNN) models for detecting fingerprinted YouTube video network traces in a novel open-world scenario
- The use of HTTPS packet network data for video traffic fingerprinting rather than low-level and more granular data as used by previous authors [2], [3], [7] and defined by the Open Systems Interconnection (OSI) model.
- The collection of the, to our knowledge, largest dataset with 6289 records including 50 network traces of 50 videos each for fingerprinting. The full dataset is available at kaggle
- A data collector that can automatically and robustly gather bitrate information from a given set of YouTube videos

We define research questions based on observations in related works. Authors like [2] achieve high performance using a simple distance-measure approach for the task of video identification. Further studies like [4] employ a more complex CNN-based approach while achieving similar results. The given task is subject to dataset shifts, a phenomenon in machine learning where the data point distribution of training and testing or deployment datasets differs [8]. Previous authors assume a closed-world scenario where all traces are known [2]. However, in an open-world scenario, most traces would be “unknown” to the model. In addition, model complexity can lead to overfitting and thus lower performance on different datasets [9]. Based on these observations, we aim to answer the following research questions:

- 1) To what extent is there an identifiable pattern in HTTPS packet data from YouTube streams?
- 2) How well does a CNN perform at predicting YouTube videos in an open-world scenario using bitrate information compared to the less complex statistical methods of logistic regression and a DTW-based model?

The remainder of the paper is structured as follows. We first provide relevant background information in section II. Section III outlines our research design and is followed by descriptions of our work and results in section IV. Subsequently, we critically evaluate and relate our work with similar works in the field in section V. Finally, section VI summarises our work and its future impact on the field of network security in streaming content delivery.

II. BACKGROUND

The following sections outline relevant background information required for undertaking the study. Detailed are DASH, the proposed threat model, as well as video-traffic and identification techniques.

A. Threat Model

Previous authors trained models on network layer data, captured by reading the device's network interface. Obtaining data at this level of granularity requires read permission to the device's network interface, which is most often restricted to the root user [10]. However, a successful Software Supply Chain (SSC) attack, in which adversaries seek to inject malicious code from third-party proprietary and open-source software used by downstream developers, could allow an adversary to exfiltrate the aforementioned metadata without requiring a foothold on the victim's network or elevated privileges [11].

B. Dynamic Adaptive Streaming Over HTTP

DASH is an implementation of Adaptive Bitrate Streaming (ABS), which seeks to maintain the user's Quality of Experience (QoE), by providing the client with video segments in different resolution levels according to the client's network conditions. [12], [2]. Figure 1 illustrates a DASH streaming session as a series of sequential client-initiated HTTP requests, for DASH segments [13].

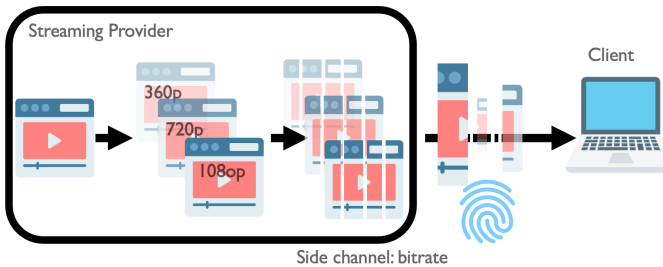


Fig. 1. DASH illustration

DASH uses the H.264 industry standard for video compression. H.264 encoding reduces the size of the digital video

through removing redundancy within and between frames among other techniques [14]. The encoding process results in a unique representation across videos based on video-specific characteristics. The correlation between encoded on-disk segments and network traffic patterns, coupled with the sequential nature of DASH, leaks a video-specific identifiable pattern even in encrypted traffic for certain videos [2].

C. Video-Traffic Fingerprinting

DASH's adaptive bitrate strategy leads to variances in traffic measures of the same video captured at different points in time. Figure 2 is exemplary of the identifiable pattern, displaying the captured HTTPS packet sizes over time of the same video at three different times. However, these traces have variances in the size of the packets and shifts in time. A video fingerprint must be able to capture a unique pattern despite these variations.

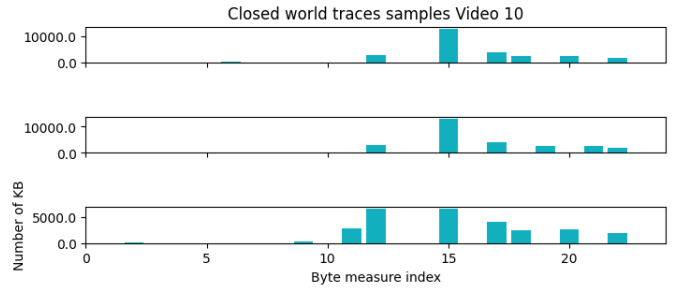


Fig. 2. Example traces of the same video

We define a video fingerprint f as the pattern that characterises the sequence of packet sizes $s = (s_1, s_2, \dots, s_n)$ associated with streaming a specific YouTube video. Several methods can be implemented to fingerprint a YouTube video.

One method to create such a fingerprint is grouping the packet sizes into periods, where each period contains all measures assumed to belong to one dash segment [3], [2]. Figure 3 illustrates this process. This can mitigate differences in time [15].

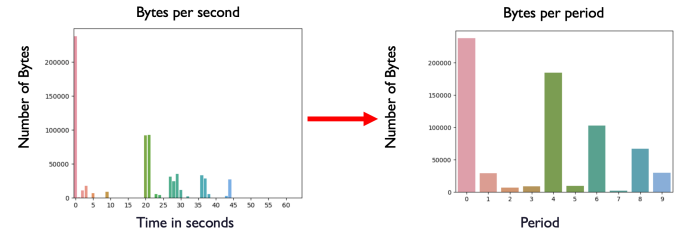


Fig. 3. Traffic trace aggregation into bytes per period

Variations in packet sizes of different traces of the same video can be mitigated by regarding the relative differences between bitrate measurements. One such method is the Differential Fingerprint (DF) described in equation 1. The fingerprint

is a sequence of fingerprint measures f_i that represent the difference between two adjacent sequence measures s_i and s_{i+1} in proportion to the first measure [3] and [2].

Other possible fingerprints include the Simple Difference Fingerprint (SDF) in equation 2, and the Absolute Difference Fingerprint (ADF) in equation 3 [3]. In addition to previous authors, we define the Magnitude Preserving Fingerprint (MPF) in equation 4, which aims to capture positive and negative differences better than the regular differential fingerprint, by normalising two adjacent measures by the larger value of the two values. Lastly, we define normalised differential fingerprint in equation 5 which produces fingerprinting values between 0 and 1.

$$f_i = F_{df}(s_i, s_{i-1}) = \frac{s_i - s_{i-1}}{s_{i-1}} \quad (1)$$

$$f_i = F_{sdf}(s_i, s_{i-1}) = s_i - s_{i-1} \quad (2)$$

$$f_i = F_{adf}(s_i, s_{i-1}) = |s_i - s_{i-1}| \quad (3)$$

$$f_i = F_{mpdf}(s_i, s_{i-1}) = \frac{s_i - s_{i-1}}{\max(s_i, s_{i-1})} \quad (4)$$

$$f_i = F_{ndf}(s_i, s_{i-1}) = \frac{s_i - s_{i-1}}{(s_i + s_{i-1})} \quad (5)$$

Since the traffic traces are time series, the fingerprint can also consist of extracted time-series features. These include the Fourier entropy, the number of duplicate values, or variance. Another option for fingerprinting is automatic pattern extraction through a CNN as in [4]. A CNN can recognise more complex features not evident to humans or outside of well-known time-series features.

D. Video Identification

Video identification involves comparing a sample of a victim's traffic to all fingerprints in the attacker's fingerprint database and producing a classification.

A Video fingerprint sequence f can be identified using similarity measures such as Dynamic Time Warping (DTW). DTW measures the similarity of two-time series by selecting the best match of a set of non-linear alignments. Thereby, the similarity measure is not sensitive to time distortions and is preferable over linear distance measures such as cross-correlation [16], [17]. [2] demonstrates the feasibility of using Partial DTW (P-DTW), which has the potential to align sequences of differing lengths and find partial alignments.

Another option for fingerprinting is employing an automatic approach using deep neural networks. [4] and [7] demonstrate the use of CNNs for automatic video identification. Although the CNN can be used for both fingerprint extraction and video identification [4], previous authors have also achieved successful classifications using hand-crafted features like an SDF as input to their CNN [3], [7].

III. METHODOLOGY

A. Data Collection

We collected a dataset of 6289 YouTube video bitrate measure traces of, including 50 traces of 50 videos for fingerprinting. Additionally, one or more traces of 3665 videos each are captured for the open-world scenario. The following described this data collection process comprised of two stages: selecting and extracting a set of valid URL uniform resource locators (URL) for videos of two or more minutes in length; and automatically collecting video traffic traces.

1) *Extracting Uniform Resource Locators:* We extracted 9000 video uniform resource locators (URL) through web-scraping the "Random YouTube Video Generator" [18]. We defined URLs for videos longer than two minutes as valid. 6000 valid URLs remained after extracting the video length with the help of Selenium a browser automation tool [19]. Next, for each video, we captured the number of bytes transmitted at a given timestamp between the user's browser and the YouTube server for two minutes. The data capture sessions are performed using Selenium, powered by the Firefox geckodriver [20]. The geckodriver was chosen to avoid the Completely Automated Public Turing test to tell Computers and Humans Apart (CAPTCHA), a prevalent occurrence when interacting with YouTube using the chromium driver [19]. Browser caching functionality was disabled across viewing sessions, to ensure the browser retrieves the entire contents of the video for each session. Finally, a Firefox profile with AdBlocker installed seeded each browser session to ensure the network traffic data pertained solely to the contents of the requested video. HTTPS-packet data is captured from the Open Source proxy.py's [21] standard output.

2) *Automated Data Extraction:* Figure 4 illustrates the data collection system architecture. The functionality necessary to retrieve N view traces of a specific URL was encapsulated in the DataHarvester Thread (DHT). The orchestrator acts as a main program which converts the set of valid URLs into a pool of DHT tasks. An upper bound of three concurrent workers was chosen after witnessing a noticeable, predictable degradation of video quality when this limit was breached.

The DHT begins by initialising the Context Managed Githandler. The Context Manager design pattern in Python allows the developer to define default behaviour upon entering and exiting the object. Upon entry, Githandler clones the streamingdatarepository repository master branch to a DHT-specific location on disk and creates a remote branch whose name is a Unique Identifier (UID) consisting of an amalgam of the URL, port and timestamp. Using a unique branch for each DHT instance, while greatly bloating the number of branches, removed the potential for fast-forward errors and merge conflicts. The DHT finishes by initialising the Harvester class. The Harvester gathers N traces of the specified URL via the Viewer and Proxy classes. The Selenium viewer session is proxied through a DHT-specific proxy instance to enforce network isolation and capture HTTPS-packet traffic. The open-source library proxy.py [21] was used for this purpose. The

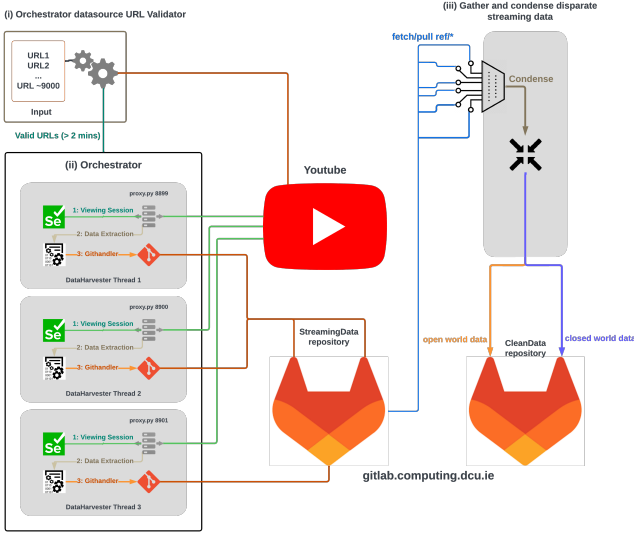


Fig. 4. DataHarvester architecture

Viewer class handles the entire interaction with YouTube via Selenium, including navigation through the various popup menus, clicking the play button, and watching the video for the specified duration. Upon terminating the viewing session, a kill signal is sent to the various proxy processes, the standard output is captured and subsequently fed into an extractor which retrieves the relevant metadata. The data is converted to a Pandas DataFrame, written to disk added, then finally added, committed, and pushed to GitLab.

Finally, the PullFetchThread iterates through the disparate Git branches created by each respective DHT and condensed into an open-world and closed-world Pandas DataFrame.

B. Data Analysis

A thorough analysis of the dataset was performed before the fingerprint detection models were created. The following sections describe the analysis methodology and findings.

1) *Inspecting Fingerprint Uniqueness*: Visual inspection and hypothesis testing reveal significant similarities between OSI high-level layer traffic traces of the same video. We use DTW-barycenter averaging to analyse patterns in traces of different videos. DTW-barycenter averaging is a method for averaging time series, that synthesises a new series that minimises the DTW-distances to each element in a set of time series [17]. Figure 5 exemplary shows the DTW-barycenter averages of three videos in our dataset (blue) and the original traces belonging to the respective video (grey).

We empirically test the video trace uniqueness by comparing the similarity of video traces stemming from one video (within-group similarity) and the dissimilarity of traces stemming from different videos (between-group similarity) and hypothesis testing. The following hypotheses were analysed.

- 1) Different traces of one video are highly correlated with a maximal value of above 0.5.

- 2) Different traces of one video are not highly cross-correlated, with a maximal value of below 0.5.

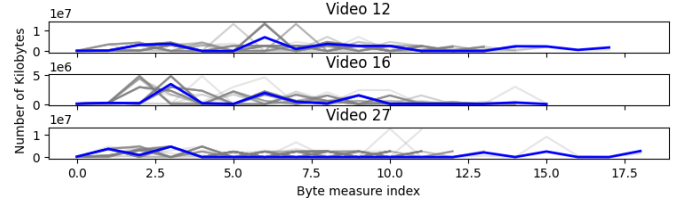


Fig. 5. Averaging fingerprints

We compare video traces using cross-correlation defined in equation 6. Cross-correlation is a similarity measure, often used to compare time series. It measures the correlation between two-time series s_i and s_j at different offsets τ at a specific time t in a time series.

$$R_{s_i s_j}(\tau) = \sum_t s_i(t) \cdot s_j(t + \tau) \quad (6)$$

For the within-group similarity, we calculate the maximum cross-correlation R from all offsets τ between samples of the same video. For the between-group similarity, we calculate the maximum cross-correlation R from all offsets τ between traces of different videos using equation 8. Due to the exponential complexity of this problem, we sample a big enough sample size from our dataset and perform t-tests.

We calculate the within-group similarity using equation 7. For each of the 50 videos, k we sample six traffic traces s_{ki} and another six different traffic traces s_{kj} . We then cross-correlate the resulting N pairs and use a t-test to analyse, if the mean of the sample is significantly above 0.5. Our test concludes, that at a 0.05% significance level, the samples are significantly highly correlated. The mean correlation of the samples is 0.77.

$$\bar{R}_{\text{within}} = \frac{1}{N} \sum_{i=1}^6 \sum_{j=1, j \neq i}^6 \sum_{k=1}^{50} \max_{\tau} (R_{s_{ki} s_{kj}}(\tau)) \quad (7)$$

Analogously to the within-group similarity we analyse the between-group similarity. For this, we choose six traces s_{ki} for each video k and cross-correlate them each with another six traces s_{lj} from differing videos. We could not conclude that the between-group differences are significantly below 0.5. The mean is 0.57. Although the traces of different videos are cross-correlated, the within-group cross-correlations are significantly higher than the between-group correlations, which leaves enough evidence to conclude the traffic traces belonging to one video are distinguishable from traces of other videos.

$$\bar{R}_{\text{between}} = \frac{1}{N} \sum_{k=1}^{50} \sum_{i=1}^6 \sum_{l=1, l \neq k}^6 \sum_{j=1}^6 \max_{\tau} (R_{s_{ki} s_{lj}}(\tau)) \quad (8)$$

2) *Binning Intervals for the Traffic Measures*: The traffic measures are not taken in steady time intervals. Similar to previous authors we analyse the effect of different discretisation binning intervals [2] on the within and between-group cohesion measures by the cross-correlation. Figure 6 shows that, unlike the previous authors, the bin size has no apparent effect on the group coherence. This can be explained by our data being less granular and already aggregated as we are capturing the packet sizes on the application level, rather than the network level.

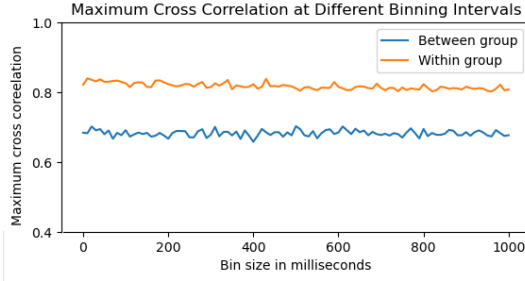


Fig. 6. Effect of bin-size on cluster coherence

3) *Fingerprinting Method*: Lastly, we analyse the effect of the different fingerprinting methods on the fingerprint group cohesion. As shown in figure 7 the DF, leads to the highest within-group cross-correlation. However, it also produces a high between-group cross-correlation. This is not ideal as it reduces group cohesion.

The SDF and the MDF were chosen for subsequent analysis and modelling as they lead to high group cohesion.

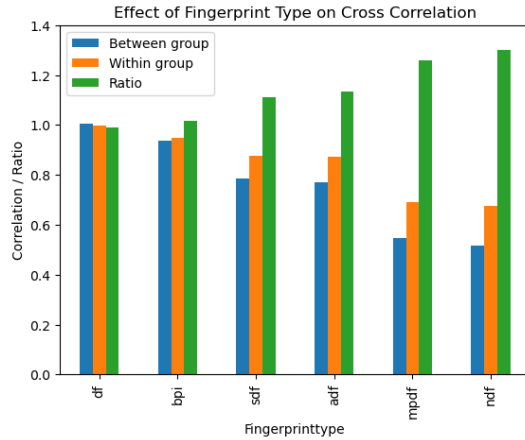


Fig. 7. Group coherence by fingerprint type

C. Data Processing

Using insights from the data analysis, we perform several data processing steps. Initially, byte size measures were discretised into one-second intervals for data and noise minimisation. This step is conducted confidentially as the interval choice has negligible impact on group cohesion. Subsequently, fingerprints, as outlined in section III.D, were derived from Bytes Per Second (BPS) measures.

Furthermore, we adapt the fingerprinting datasets for the models specified in section III.D, necessitating uniform data length for logistic regression and CNN models, resulting in two dataset variations. The first adjusts all fingerprints to the length of the longest one via padding, while the second constructs a 120-element array representing a two-minute capture period, with BPS data assigned accordingly.

In alignment with the open-world scenario in section I, we partition 2500 fingerprint traces into training and testing sets with an 80/20 split, maintaining class proportion equality. To simulate open-world conditions, 3000 and 662 open-world traces were integrated into the test and training datasets, respectively. The training set's number of "unknown" samples varies by model. Models were developed using TensorFlow, scikit-learn, and tsfresh.

D. Video Detection Models

We employ and compare three models for video identification in our study, a CNN, a logistic regression, and a P-DTW-based model. We use these models, to validate and improve upon previously deployed models. We also test their performance in a closed and open-world setting. Here, the models must correctly reject the large number of unknown traces with high recall, yet confidently identify known traces with high precision and recall. Therefore, models all models are evaluated using the macro-averaged F1 metric as it balances the recall and precision without bias towards a majority class.

1) *Logistic Regression*: Using the tsfresh Python library for automatic feature extraction and selection, we explored multinomial and one-vs-rest logistic regression strategies [22]. To accommodate "unknown" samples, the multinomial approach, was deemed unsuitable as it assumes all potential classes are known. Consequently, a one-vs-rest logistic regression was adopted to classify samples into one of N classes or as "unknown" in both open and closed-world scenarios. In the open-world setting, a threshold of 0.2, selected for its optimal performance, was used to deem predictions not confidently assigned to one of 50 classes as "unknown". Tsfresh enabled the extraction and selection of 370 features, such as autocorrelation lags, from the dataset. Subsequent analysis identified significant multicollinearity, as evidenced by high variance inflation factors. This violates the logistic regression prerequisite for independent variables and diminishes the significance assumption of independent features [23]. Therefore, we eliminated these collinear features yielding remaining 72 features for modelling. The model achieved optimal performance using these 72 features, along with an L2 regularization and an inverse regularization strength of 0.1, across various datasets, including MPFP and SDF.

2) *Dynamic Time Warping Model*: Adopting the methodology of [2], we implement their P-DTW-based approach for video-traffic and identification, constructing a database of MDFP and SDF fingerprints. Prediction involves matching a transformed traffic trace to the closest fingerprint within this database. Unlike the original study, which generates a singular fingerprint per class from a single traffic trace, our approach

requires selecting an optimal fingerprint from multiple traces. We hypothesise that fingerprints derived from longer traces with greater variance contain more identifiable information, thus enhancing prediction accuracy. Additionally, we explore the efficacy of fingerprint averaging. To validate our hypotheses, we select the ten traces with the highest variance from a set of 50 videos, synthesizing averaged fingerprints from the one to ten highest variance traces using barycenter averaging. This process is replicated for the ten longest traces. Our findings indicate that trace length does not correlate with improved prediction accuracy; however, averaging the ten highest variance traces improves results.

3) *Convolutional Neural Network*: We follow [3] and [7] and recreate a CNN for video-traffic fingerprinting and identification. Both the 120-dimensional vector as well as the zero-padded fingerprint measures for the MPDF and SDF were tested. The best performance was reached with the SDF. To prevent overfitting, we increased the CNN's drop-out rate to 0.9, reduced the number of epochs to 100, and used early stopping.

IV. RESULTS

As shown in table I of all tested models the CNN performs best in the open and closed-world scenario. With an F1-score of 80%, it outperforms itself in the open-world scenario compared to the closed-world scenario. Further, it distinguishes between known and unknown traces with 92% recall. The logistic regression outperforms the DTW model, both in the open and closed-world scenario. The DTW model performs worst out of all tested models, with an F1-score of 17%.

TABLE I
MODELS TEST SCORES

Closed-World			
Model	DTW	LR	CNN
F1-score	0.39	0.63	0.79
Accuracy overall	0.46	0.64	0.78
Open-World			
Model	DTW	LR	CNN
F1-score	0.17	0.31	0.80
Accuracy overall	0.18	0.75	0.78
Recall negatives	0.12	0.27	0.92

V. DISCUSSION

Our work is related to previous works in the field of video-traffic in several aspects. We followed the CNN architectures of [3]. With minor changes to the architecture, we were able to validate their results on a new dataset and confirm their model works comparably well in an open-world scenario.

We based our DTW model on the works of [2], yet we could not replicate their results in the open or closed-world scenarios. This could potentially be due to an inherent bias towards low-variance fingerprints of the model. Our model predicts traces with a low variance significantly more often than traces with a high variance. This, however, is only a correlation, not causation, and requires further research. Although the model

performs worse out of all tested models, with an F1-score of 17% it outperforms the baseline of 1/51 or 2%.

To our knowledge, we are working with the largest dataset for the task of YouTube video-traffic in the reviewed literature. Only [7] employed an open-world scenario, and we were able to improve upon their results. They claim a false positive rate of 0 however, this only applies to 20% of their data. They could not extract an identifiable fingerprint from the remaining data. Although our open-world study design simulates a realistic scenario, it has to be yet tested in a real-world setting.

To our knowledge, we are the first in the field to employ a logistic regression for the task of YouTube video identification. However, the model's performance was significantly reduced in the open-world scenario, despite implementing several regularisation and feature engineering techniques. Options for improving the model could include adding higher-order terms to discover such a relationship or using a non-linear model such as a k-nearest neighbours (KNN). We did not examine further popular machine learning models such as Random Forest [24] as we aimed to examine whether less complex models used by similar authors' works performance can reach their claimed performance on a different dataset, which would render a CNN approach potentially surfeit.

VI. CONCLUSION AND FUTURE WORK

This study examined the feasibility of a side-channel video-traffic-fingerprinting attack with higher OSI-layer level data extracted from encrypted network traces leading to the following conclusions.

We answered our first research question, by demonstrating that there is an identifiable pattern in high-level network traces which we were able to extract and identify with all proposed models. All models performed better than the baseline of a random classifier with an F1-score of 2%. We answered our second research question by demonstrating that the CNN model performs best in the open and closed-world scenarios with an F1-score of 80% in the closed-world and a recall of 92% on the "unknown" class. Contrary to our initial belief, this shows, that the CNN model effectively extracts patterns from a relatively small number of samples per class. Thereby, the model effectively handles dataset shifts. Our CNN model performance confirms the work of [3], by reaching comparable results using higher-level data. In contrast, we were not able to reach similar performances with our statistical models. Logistic regression, as well as the DTW, fail to generalise on unrelated samples of the open-world, thus, are not suited for a real-world implementation of the attack. This study examined a limited number of algorithms for the task of video-traffic fingerprinting. Future works could compare and examine more algorithms. Finally, we collected an extensive dataset containing more than 6000 open and closed-world traces that we made openly available for further research.

In conclusion, our study confirmed the feasibility of an SCNN-attack on encrypted network traces that can infringe on the privacy of streaming content consumers. This calls for further research on protecting against such an attack.

REFERENCES

- [1] Michail Michalos, Stelios Kessanidis, and S.L. Nalmpantis. “Dynamic Adaptive Streaming over HTTP”. In: *Journal of Engineering Science and Technology Review* 5 (June 1, 2012), pp. 30–34. DOI: 10.25103/jestr.052.06. (Visited on 04/25/2023).
- [2] Jiayi Gu et al. “Traffic-Based Side-Channel Attack in Video Streaming”. In: *Ieee-Acm Transactions on Networking* 27.3 (June 1, 2019), pp. 972–985. DOI: 10.1109/TNET.2019.2906568. (Visited on 10/03/2022).
- [3] Waleed Afandi et al. “Fingerprinting Technique for YouTube Videos Identification in Network Traffic”. In: *IEEE Access* 10 (July 19, 2022), pp. 76731–76741. DOI: 10.1109/ACCESS.2022.3192458. (Visited on 03/04/2023).
- [4] Muhammad U. S. Khan et al. “SCNN-Attack: A Side-Channel Attack to Identify YouTube Videos in a VPN and Non-VPN Network Traffic”. In: *Electronics* 11.3 (3 Jan. 2022), p. 350. DOI: 10.3390/electronics11030350. (Visited on 10/02/2022).
- [5] Luming Yang et al. “Markov Probability Fingerprints: A Method for Identifying Encrypted Video Traffic”. In: *2020 16th International Conference on Mobility, Sensing and Networking (Msn 2020)*. 2020, pp. 283–290. DOI: 10.1109/MSN50589.2020.00055. (Visited on 10/21/2022).
- [6] Vera Rimmer et al. “Automated Website Fingerprinting through Deep Learning”. In: *Proceedings 2018 Network and Distributed System Security Symposium*. Feb. 17, 2018. DOI: 10.14722/ndss.2018.23105. (Visited on 10/05/2022).
- [7] Roei Schuster, Vitaly Shmatikov, and Eran Tromer. “Beauty and the Burst: Remote Identification of Encrypted Video Streams”. In: *Proceedings of the 26th USENIX Conference on Security Symposium*. 2017, pp. 1357–1374. ISBN: 978-1-931971-40-9. (Visited on 12/30/2022).
- [8] Storkey Amos. “23When Training and Test Sets Are Different: Characterizing Learning Transfer”. In: *Dataset Shift in Machine Learning*. The MIT Press, Dec. 2008. ISBN: 9780262170055. DOI: 10.7551/mitpress/9780262170055.003.0001. eprint: https://academic.oup.com/mit-press-scholarship-online/book/0/chapter/166918395/chapter-ag-pdf/44903282/book/_13447/_section/_166918395.ag.pdf. URL: <https://doi.org/10.7551/mitpress/9780262170055.003.0001>.
- [9] Xia Hu et al. “Model Complexity of Deep Learning: A Survey”. In: *Knowledge and Information Systems* 63 (Oct. 1, 2021). DOI: 10.1007/s10115-021-01605-0. (Visited on 07/26/2023).
- [10] IBM. *tcpdump*. <https://www.ibm.com/docs/zh/aix/7.1?topic=t-tcpdump-command>. 2023. (Visited on 06/22/2023).
- [11] Betul Gokkaya, Leonardo Aniello, and Basel Halak. *Software supply chain: review of attacks, risk assessment strategies and security controls*. 2023. arXiv: 2305.14157. (Visited on 06/22/2023).
- [12] Kamila Ragimova, Vyacheslav Loginov, and Evgeny Khorov. “Analysis of YouTube DASH Traffic”. In: *2019 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*. 2019-06-03, pp. 1–5. DOI: 10.1109/BlackSeaCom.2019.8812782. (Visited on 04/23/2023).
- [13] Abdelhak Bentaleb et al. “A Survey on Bitrate Adaptation Schemes for Streaming Media Over HTTP”. In: *IEEE Communications Surveys Tutorials* 21.1 (2019), pp. 562–585. DOI: 10.1109/COMST.2018.2862938. (Visited on 02/23/2023).
- [14] Humberto de Jesus Ochoa Dominguez et al. “The H.264 Video Coding Standard”. In: *IEEE Potentials* 33.2 (2014), pp. 32–38. DOI: 10.1109/MPOT.2013.2284525. (Visited on 03/22/2023).
- [15] Christopher Mueller. *MPEG_DASH Dynamic Adaptive Streaming Over HTTP — Bitmovin*. URL: <https://bitmovin.com/dynamic-adaptive-streaming-http-mpeg-dash/> (visited on 07/31/2023).
- [16] Z. Zhang et al. “Dynamic time warping under limited warping path length”. In: *Information Sciences* 393 (2017), pp. 91–107. DOI: 10.1016/j.ins.2017.02.018.
- [17] François Petitjean, Alain Ketterlin, and Pierre Gançarski. “A Global Averaging Method for Dynamic Time Warping, with Applications to Clustering”. In: *Pattern Recognition* 44.3 (Mar. 2011), pp. 678–693. DOI: 10.1016/j.patcog.2010.09.013. (Visited on 05/03/2023).
- [18] *Random YouTube Video Generator — Perchance*. URL: <https://perchance.org/youtube-video> (visited on 02/23/2024).
- [19] Jason Huggins. *geckodriver*. <https://github.com/SeleniumHQ/selenium>. 2004.
- [20] mozilla. *geckodriver*. <https://github.com/mozilla/geckodriver>. 2022.
- [21] Abhinav Singh. *proxy.py*. <https://github.com/abhinavsingh/proxy.py>. 2019. (Visited on 06/22/2023).
- [22] Romain Tavenard et al. “Tslearn, A Machine Learning Toolkit for Time Series Data”. In: *Journal of Machine Learning Research* 21.118 (2020), pp. 1–6. URL: <http://jmlr.org/papers/v21/20-091.html> (visited on 06/12/2023).
- [23] Jill C. Stoltzfus. “Logistic Regression: A Brief Primer: LOGISTIC REGRESSION: A BRIEF PRIMER”. In: *Academic Emergency Medicine* 18.10 (Oct. 2011), pp. 1099–1104. DOI: 10.1111/j.1553-2712.2011.01185.x. URL: <https://onlinelibrary.wiley.com/doi/10.1111/j.1553-2712.2011.01185.x> (visited on 07/29/2023).
- [24] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32. ISSN: 08856125. DOI: 10.1023/A:1010933404324. URL: <http://link.springer.com/10.1023/A:1010933404324> (visited on 05/15/2024).